

软件工程基础

第 2 章—软件过程

学习笔记

2026 年 6 月 1 日

来源：软件工程基础第二章课件（125 页） 覆盖模型：瀑布 | 原型 | RAD | 增量 | 螺旋 | RUP | XP | Scrum | 精益 | FDD

目录

1	软件过程概述	2
1.1	软件过程与软件生存期过程	2
1.2	国标 GB/T 8566 演进	2
1.3	软件生存周期模型（Software Life Cycle Model）	2
2	传统软件过程模型	3
2.1	瀑布模型（Waterfall Model）	3
2.2	快速原型模型（Prototyping Model）	3
2.3	快速应用开发模型（RAD, Rapid Application Development）	4
2.4	渐增模型 / 增量模型（Incremental Model）	5
2.5	螺旋模型（Spiral Model）	6
2.6	统一过程模型（UP / RUP, Unified Process / Rational Unified Process）	7
2.7	六种传统过程模型统一对比	9
3	敏捷开发（Agile Development）	11
3.1	敏捷开发背景	11
3.2	敏捷联盟宣言（The Manifesto of the Agile Alliance）	11
3.3	12 条敏捷原则（12 Agile Principles）	11

4 极限编程 (Extreme Programming, XP)	13
4.1 XP 五大价值观 (Five Values of XP)	13
4.2 XP 原则与实践概览	13
4.3 XP 完整过程	13
4.3.1 阶段一：规划 (Planning)	13
4.3.2 阶段二：迭代 / 构建迭代 (Iteration)	15
4.3.3 阶段三：测试 (Testing)	17
4.3.4 阶段四：发布 (Release)	17
5 Scrum	18
5.1 三大角色 (Three Roles)	18
5.2 冲刺与 PDCA 循环	18
5.3 三大制品 + 燃尽图 (Artifacts)	19
5.4 四种会议 (Four Meetings / Ceremonies)	19
5.5 冲刺计划计算示例	20
6 精益开发与特征驱动开发	21
6.1 精益软件开发 (Lean Software Development)	21
6.2 特征驱动软件开发 (FDD, Feature Driven Development)	21
7 敏捷项目管理工具	23
7.1 PingCode	23
7.2 Gitee 企业版	23
附录：中英术语对照表	24

1 软件过程概述

1.1 软件过程与软件生存期过程

- **软件过程 (Software Process):** 软件产品生产由一组相互有机联系的活动来完成, 这些相互有机联系的活动便构成软件过程。软件过程决定了软件质量, 开发依据过程, 管理选择过程模型。 [p.4]
- **软件生存期过程模型:** 软件生存期各项活动/任务/子过程的有机结合, 以满足特定项目需要。 [p.6]
- 软件生存期过程规定了获取、供应、开发、操作和维护软件时要实施的过程、活动和任务。其目的是为各种人员提供公共框架, 以便使用“相同的语言”创作和管理软件。 [p.6]
- 软件生存期过程没有规定一个特定的生存周期模型, 各软件开发机构可视项目需要选择一种软件生存周期模型, 并将过程、活动和任务映射到选定的模型中。 [p.6]

1.2 国标 GB/T 8566 演进

表 1: GB/T 8566 标准版本对比

版本	内容划分	说明
GB/T 8566-1995	7 个基本生存期过程: 获取、供应、管理、开发、操作、维护、支持	早期版本 [p.8]
GB/T 8566-2007	5 个基本过程 + 9 个支持过程 + 7 个组织过程	更细化、更全面的分类[p.9-12]

1.3 软件生存周期模型 (Software Life Cycle Model)

- 亦称软件开发模型, 描述了从需求定义开始, 到开发成功投入使用, 在使用中不断增补修订, 直到停止使用, 这一期间各种活动如何执行的模型。 [p.17]
- 开发机构应综合项目性质、应用性质、方法工具等, 选择合适的模型或构建, 并将软件生存期过程映射到选定的模型中进行开发和维护。 [p.17]

2 传统软件过程模型

2.1 瀑布模型（Waterfall Model）

瀑布模型由温斯顿·罗伊斯（Winston Royce）于 1970 年提出，也称为传统生命周期模型。[p.19]

表 2: 瀑布模型概要

方面	内容
核心特征	- 各阶段顺序相互依赖，形成软件生产流水线（软件产业化、职业化） - 每阶段生成文档并进行评审：文档驱动，软件过程可视化，质量保障 - 强调需求分析和设计，推迟物理实施（有助于提高质量） [p.20]
优点	- 阶段清晰，便于管理和控制 - 文档驱动，过程可视化 - 强调早期需求分析和设计，质量有保障
缺点	- 难以适应需求模糊的情况（开发前期用户需求模糊不全面） - 开发过程中用户看不见系统，得不到反馈 - 需求不确定或快速变化时，返工代价大，难以适用非线性开发 [p.21]
适用场景	功能和性能需求明确的软件项目，如编译器、操作系统等 [p.20-21]

线性顺序结构：需求分析 → 设计 → 编码 → 测试 → 维护，各阶段严格顺序推进，不可回溯。

2.2 快速原型模型（Prototyping Model）

原型是一个可以实际运行的模型，在功能上可看作最终产品的子集，展示目标系统的关键功能。[p.23]

表 3: 快速原型模型概要

方面	内容
核心特征	• 通过向用户提供原型获取反馈（“百闻不如一见”） • 采用逐步求精方法完善原型，避免冗长的开发过程 • 快速开发原型，便于快速响应用户意见 • 不断迭代改进，更符合人们开发软件的习惯 [p.24]
优点	• 有效获取真实用户需求 • 快速响应用户反馈 • 减少需求误解风险
缺点	• 不宜直接利用原型系统作为最终产品（原型成本问题） • 原型对软件异常等问题考虑较少 • 快速特点对最终系统不适用（原型语言和工具与最终系统不同） • 用户和开发者必须达成一致：原型仅用于定义需求，之后部分或全部抛弃 [p.25]
适用场景	需求模糊或不确定的软件项目 [p.25]

迭代结构：快速分析 → 构造原型 → 运行原型 → 评价原型 → 修改意见，循环迭代直至需求明确，最终开发正式产品。

2.3 快速应用开发模型（RAD, Rapid Application Development）

RAD 模型是线性开发模型，特别强调极短的开发周期（如 2-3 个月）。[p.27]

表 4: RAD 模型概要

方面	内容
五个阶段	• 业务建模 (Business Modeling): 弄清业务活动中的信息流 • 数据建模 (Data Modeling): 精化业务建模结果 • 处理建模 (Process Modeling): 依据数据建模结果, 创建处理描述 • 应用生成 (Application Generation): 组件复用与开发 • 测试 (Testing): 测试新组件及所有接口 [p.28]
优点	• 极短开发周期 (2-3 个月) • 强调组件复用, 提高效率
缺点	• 需要足够的人力创建足够的 RAD 小组 • 技术风险高的情况不适合 (如需要高互操作性、系统难以划分为独立组件时) • 开发者和用户需在很短时间内完成系统开发 [p.29]
适用场景	主要用于信息系统应用软件的开发 [p.28]

2.4 渐增模型 / 增量模型 (Incremental Model)

演化模型都是迭代增量式的, 使软件工程师能够开发出越来越完善的软件版本。渐增模型由若干轮开发过程组成, 每一轮都是在上一轮基础上进行。

[p.31]

表 5: 渐增模型概要

方面	内容
核心特征	• 结合瀑布模型的直线式特点和快速原型化模型的迭代思想 • 每一轮开发过程是直线式的，但整体在前一轮基础上迭代 • 每一轮都是产品（区别于原型模型：原型模型每轮得到的是原型） • 不需要大的资金支出，投资回报随功能渐增而渐增 • 用户能及早使用、及早发现问题 • 能够有计划地管理技术风险 • 可根据需要补充人员 [p.32]
优点	• 早期交付核心功能，快速获得投资回报 • 减少全新产品对用户的冲击 • 有计划地管理技术风险 • 人员可灵活调配
缺点	• 对设计水平要求高（整体结构设计不当则难以增加新增量） • 难于进行彻底测试（有些增量在后期开发） [p.33]

2.5 螺旋模型（Spiral Model）

通过使用原型或其他类似手段以降低风险的思想，是螺旋模型中蕴涵的基本思想。最早由巴利·玻姆（Barry Boehm）于 1988 年提出。

[p.35]

表 6: 螺旋模型概要

方面	内容
核心特征	• 风险驱动: 风险分析使用户和开发者更好地理解对待每个阶段的风险，目的是“去风险” • 结合诸多模型特点: 保持系统阶段性方法 + 通过建造原型排除风险 + 自内向外逐步延伸（体现渐增模型特点） [p.36]
四象限结构	• 第一象限: 确定目标、方案和约束 • 第二象限: 评估方案、识别并排除风险（风险分析） • 第三象限: 开发和验证下一级产品（工程实现） • 第四象限: 计划下一阶段（规划） 每轮螺旋都经历这四个象限，自内向外逐步延伸。
优点	• 风险驱动，适合高风险项目 • 融合多种模型优势 • 逐步延伸，灵活性强
缺点	• 要求开发人员擅长风险分析（重大风险未被识别、非风险当成风险） • 风险分析可能导致项目终止和合同违约诉讼（一般适合内部大系统开发） • 风险分析增加项目成本（对小项目可能成本相当） [p.37]
适用场景	大型软件开发，尤其内部创新性项目 [p.37]

2.6 统一过程模型（UP / RUP, Unified Process / Rational Unified Process）

统一过程（UP）模型是 Rational 软件公司（现已被 IBM 收购）开发的基于统一建模语言（UML）的迭代增量式软件开发模型。RUP 是 UP 的一个商业版本。

[p.39]

表 7: UP / RUP 模型概要

方面	内容
三大特点	- 用例驱动（Use Case Driven）：用例是系统外部参与者完成的一系列动作，确定了开发目标并驱动每次迭代的工作 - 以体系结构为中心（Architecture-Centric）：体系结构提供指导迭代的结构，帮助涉众宏观认识系统，有助于软件重用 - 迭代增量式（Iterative and Incremental）：适应需求变化，使项目计划更加灵活 [p.45]
四个阶段	- 初始阶段（Inception）：确定项目范围和业务案例 - 精化阶段（Elaboration）：细化需求，建立体系结构基线 - 构建阶段（Construction）：完成系统开发 - 移交阶段（Transition）：将系统交付用户 各阶段均产生相应制品 [p.40–43]
workflow	业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、项目管理、环境 [p.44]
优点	- 用例驱动，贴近用户需求 - 以架构为中心，支持大型复杂系统 - 迭代增量，适应变化
缺点	- 太复杂：未给出具体裁剪方法 - 太重：各阶段需开发大量制品，不适合资源有限且面临时间压力的项目 [p.46]

2.7 六种传统过程模型统一对比

表 8: 六种传统软件过程模型对比总表

模型	提 出 者/时间	驱 动 方 式	核心机制	开发周期	适用场景
瀑布	W. Royce, 1970	文 档 驱 动	线性顺序, 阶段审批	长	需求明确的系统软件（编译器、OS）
原型	—	需 求 驱 动	快速构建、用户反馈、逐步求精	中	需求模糊的项目
RAD	—	组 件 驱 动	业务 → 数据 → 处理 → 生成 → 测试	极短 (2-3 月)	信息系统应用
增量	—	功 能 驱 动	逐轮迭代, 每轮交付产品增量	中	核心功能可早期交付的项目
螺旋	B. Boehm, 1988	风 险 驱 动	四象限循环, 风险分析为核心	长	大型、创新性内部项目
RUP	Rational/IBM 案例 +	架 构 驱 动	四阶段迭代, 9 个工作流	长	大型复杂系统

表 9: 模型结构特征对比

模型	结构特征	关键不足
瀑布	严格线性序列，不可回溯	需求必须预先确定，返工代价大
原型	循环迭代（分析-构造-评价）	原型质量不足，可能误导用户
RAD	线性但组件化	需要大量人力，不适合高互操作性需求
增量	多轮直线式 + 整体迭代	设计要求高，后期测试困难
螺旋	风险驱动的四象限螺旋	风险分析成本高，依赖专家
RUP	二维结构（阶段 × 工作流）	太重太复杂，需大量制品

3 敏捷开发（Agile Development）

3.1 敏捷开发背景

- 基于计划或描述的软件开发模型不仅费用昂贵，而且由于外部环境的变化造成返工非常普遍。 [p.51–52]
- 受“敏捷制造”启发，总结迭代增量式软件开发经验，提出敏捷价值观、原则和实践。 [p.52]
- 敏捷强调快速响应变化、有效沟通、客户参与、自组织团队。 [p.55]

3.2 敏捷联盟宣言（The Manifesto of the Agile Alliance）

2001 年 2 月，Kent Beck 和其他 16 位软件专家成立了“敏捷联盟（Agile Alliance）”，共同签署了敏捷联盟宣言。敏捷价值观把人的因素放在第一位。 [p.54]

表 10: 敏捷宣言：4 条价值观

我们更重视……（左项），尽管右项也有价值	
1.	个体和互动（Individuals and Interactions）高于流程和工具
2.	可工作的软件（Working Software）高于详尽的文档
3.	客户合作（Customer Collaboration）高于合同谈判
4.	响应变化（Responding to Change）高于遵循计划

3.3 12 条敏捷原则（12 Agile Principles）

敏捷联盟提出的 12 条敏捷原则是敏捷实践区别于传统过程模型的重要特征。 [p.57–58]

表 11: 12 条敏捷原则（分类：物 = 交付/技术，人 = 组织/团队）

#	原则内容	维度	关键词
1	最优先要做的是通过尽早地、持续地交付有价值的软件来使客户满意	物	价值、客户满意
2	即使在开发后期，也欢迎需求改变。敏捷过程利用变化为客户创造竞争优势	物	需求、竞争优势
3	经常性地交付可以工作的软件，交付间隔从几周到几月，越短越好	物	交付、短周期
4	在整个项目开发期间，业务人员和开发人员必须天天在一起工作	人	合作、日常沟通
5	围绕有积极性的个人构建项目团队，提供所需环境和支持，信任他们	人	个人、信任
6	在团队内部，最有效果并富有效率的传递方法是面对面的交流	人	交流、面对面
7	可运行的软件是首要的进度度量标准	物	进度、可运行软件
8	敏捷过程提倡可持续的开发速度（责任人、开发者、用户保持长期稳定速度）	人	速度、可持续
9	持续关注优秀的技能和好的设计，增强敏捷能力	人	能力、设计
10	简单（是不必做的工作最大化的艺术）是必要的	物	简单、最大化不必要工作
11	最好的架构、需求和设计出自于自组织的团队	人	团队、自组织
12	每隔一段时间，团队应反省如何才能有效地工作，并调整自身行为	人	反省、持续改进

原则分类理解

[p.58]:

- **组织方面（人）：**客户满意、竞争优势、积极性个人、优秀技能、欢迎改变、面对面交流、自组织团队、反省、客户参与、可持续速度
- **技术方面（物）：**需求变化、好的设计、简单实现、尽早持续交付、短交付周期、可工作的软件、有价值的软件

4 极限编程 (Extreme Programming, XP)

极限编程由 Kent Beck、Ward Cunningham 和 Ron Jeffries 等人通过整理优秀团队的共同之处而提出的敏捷过程。 [p.60]

- 所谓“极限”——尽力而为，然后处理其结果
- 专注于编程技术、清晰沟通和团队协作
- 只需做能为客户创造价值的事情
- 适用于任何规模的团队，适合模糊或快速变化的需求 [p.60]

4.1 XP 五大价值观 (Five Values of XP)

表 12: XP 五大价值观

价值观	英文	内涵
交流	Communication	创造团队意识和高效合作意识的最有效手段
简单	Simplicity	集中精力满足当前需要，减少交流内容，保持代码简洁，去掉不需要的需求
反馈	Feedback	修正错误，有益于获取简单的解决方案
勇气	Courage	敏捷开发必备基本要素：需要勇气保持简单、进行交流、面对错误修正
尊重	Respect	彼此关心、相互理解

[p.62]

4.2 XP 原则与实践概览

XP 原则是一组领域特定的、用于搜寻与 XP 价值观相协调的实践指南。一方面用于理解 XP 实践，另一方面可依据原则补充新实践。 [p.63]

通过在软件开发过程中运用 XP 实践来实现 XP 的价值。 [p.64–65]

4.3 XP 完整过程

XP 过程将各种极限编程实践进行有效结合。 [p.66]

4.3.1 阶段一：规划 (Planning)

规划阶段从架构探索和初始用户故事开始，开发人员通过与客户交流，尽量确定出真正重要的系统将如何工作的故事。 [p.67]

规划阶段工作：

1. 编写足够多的用户故事 (User Stories)，以定义成功的产品
2. 做必须的探索实验 (Spike)，以便对用户故事进行估算
3. 估算每一个故事实施的时间
4. 依据业务价值和时间为发布选择故事

[p.67]

系统隐喻 (System Metaphor) 在用户故事和对潜在解决方案探索的基础上，开发人员决定系统架构。为了便于理解架构，开发人员往往构建一个**系统隐喻**。 [p.68–69]

- 例 1: 基于本体的信息检索系统 → 程序工作就像一群蜜蜂，外出寻找花粉，并将花粉带回蜂巢
 - 例 2: 文本输出系统 → 使用缓冲区 (Buffer)，满时交换到磁盘，快空时换回程序继续运行
- Development without an overall design – common understanding by means of metaphor.* [p.68]

用户故事 (User Stories) 用户故事描述了从用户角度来看系统认为有价值的任务。 [p.70–71]

用户故事四要素：

- **可估算 (Estimable)**：故事不能太大，且开发人员具备开发技术
- **可测试 (Testable)**：可以验证对错（如“用户界面友好”就不可测试）
- **可完成 (Achievable)**：一个迭代周期（如 2 周）内能完成
- **可沟通 (Communicable)**：一次不需要引入太多细节，细节在迭代时产生

[p.70]

用户故事一般由两部分组成：书面卡片（客户描述的输入/输出/特征/功能等）和开发人员与客户之间的对话记录。 [p.71]

故事点估算 (Story Point Estimation) 故事点 (Story Point)：团队定义开发时间的模糊单位，用于衡量开发故事的工作量。 [p.72]

估算方法：

- **扑克法 (Planning Poker)**：
 - 每位成员得到一套带数字的卡片：0, 0.5, 1, 2, 3, 5, 8, ...
 - 用户讲解用户故事 → 团队成员质疑 → 用户解答
 - 选择一个较小的用户故事作为基准故事，故事点 = 1
 - 各成员将故事与基准比较，得出相对点数
 - 结果处理：求平均取最接近卡片数字；如结果不同则讨论后再次估算（不超过三次）
- **三角测量法 (Triangulation)**：将被估算故事与其他多个故事的相对关系做比较

开发速度 (Velocity)：总故事点 ÷ 总完成时间。

[p.72]

4.3.2 阶段二：迭代 / 构建迭代 (Iteration)

在规划阶段就发布计划达成一致后，通过迭代方式增量式地完成发布计划。 [p.73]

迭代周期 即使本次迭代周期内未完成所有用户故事，迭代也要在先前指定的日期前结束。客户和开发人员依据已完成用户故事的估算值计算本次迭代的项目速度（完成的用户故事点数），用于计算下一次迭代的周期或用户故事点数。 [p.74]

制定迭代计划 (Iteration Planning)

1. 开发人员将用户故事分解成任务 (Task)：一个开发人员在 4-16 小时内能实现的功能
2. 开发人员逐个签订 (Sign up) 他们想要实现的任务
3. 依据最近一次迭代中所完成的任务点数来确定本次可签订的任务点数
4. 任务选择持续到所有任务都分配出去，或所有开发人员都已签满
5. 若还有任务未分配，则协商，或要求客户去掉一些任务/用户故事
6. 若所有任务已分配但仍有开发人员能签订任务，则向客户要求更多用户故事

[p.75]

表 13: XP 迭代阶段核心实践

实践	说明
测试驱动开发 TDD (Test-Driven Development)	- 在开发产品代码之前，先编写测试代码，然后只编写使测试通过的产品代码 - 测试代码三大作用：验证（确保产品正确）、设计（接口设计，功能实现和使用分离）、文档（告诉他人如何使用代码） - TDD 循环：明确功能 → 编写测试 → 编写产品代码使测试通过 → 必要时重构 → 重复 [p.79]
结对编程 (Pair Programming)	- 两人在一台电脑上完成代码编写：一人主写代码，另一人评审 - 好处：更早发现错误、有利于团队能力提升、提高编程效率、有利于代码共有、增进团队凝聚力 [p.80]
重构 (Refactoring)	- 在不改变代码行为的前提下，对代码内部结构进行渐进式改进 - 通过测试确保行为不变 - 作用：使代码永远保持活力、代码清洗、应对变化 [p.81]
持续集成 (Continuous Integration)	- 将开发者的工作副本每天多次合并到主本的活动 - 完成工作：单元测试、代码检测、编译构建，甚至包括验收测试和自动部署 - 优势：提高工作效率（自动化重复劳动）、更快修复错误（早集成早发现）、更快交付成果（减少手工错误和等待时间） - 工具：Jenkins、Cruise Control 等 [p.82]
站立式会议 (Stand-up Meeting)	- 团队集中讨论当前工作，寻求解决建议 - 三个问题：自昨天开始做了什么？今天将做什么？阻碍迭代目标的是什么？ - 目的：相互交流学习，了解项目进度 [p.77]
代码集体所有 (Collective Code Ownership)	团队中任何人对任何代码都可以修改，鼓励每个人为项目各部分贡献力量 [p.78]

核心开发实践

4.3.3 阶段三：测试（Testing）

- 单元测试（Unit Test）：自动化测试
- 验收测试（Acceptance Test）：黑盒测试，由客户负责功能测试
- 所有测试在设计/编码前创建（测试先行）
- 所有单元测试必须 100% 通过
- 每个用户故事转化为若干测试
- 为使系统具有可测试性，需在系统架构层面对系统解耦

[p.83]

4.3.4 阶段四：发布（Release）

- 强调小规模、频繁地发布代码和测试
- 小型发布：尽早为客户提供业务价值，增加信心；通过反馈指导项目成功
- 如需文档，在版本趋于稳定时撰写：
 - 系统文档（技术架构、高层次需求、关键设计决策总结、设计模型等）
 - 系统操作文档（依赖关系、与其他系统交互特性、预期负载等）
 - 系统支持文档（问题参考信息、疑难上报流程、维护团队联系等）
 - 用户文档（参考手册、用户指南、支持指南、培训资料等）

[p.84]

5 Scrum

Scrum 是一种迭代增量式软件开发过程，像橄榄球赛的争球过程：快速、自组织和自适应性。[p.86]

5.1 三大角色（Three Roles）

表 14: Scrum 三大角色

角色	英文	职责
产品负责人	Product Owner	定义和维护“产品待办事项表”（Product Backlog），负责最大化产品及团队工作价值，代表利益相关者利益
Scrum 主管	Scrum Master	确保团队遵循 Scrum 理论、实践和规则，通过指导和引导使团队更高效地创建高质量产品
开发团队	Development Team	在每个冲刺（Sprint）结束交付潜在可发布的“已完成”产品增量，只有开发团队才能交付产品增量

[p.87]

开发团队特征[p.88]:

- 团队大小：7±2 人，保证灵活性同时能完成有意义的任务
- 自组织：没有人（包括 Scrum 主管）告诉开发团队如何把产品待办事项表变成可发布的功能
- 跨功能：团队整体拥有创造产品增量所需的全部技能
- 无头衔：无论承担哪种工作，他们都是开发者（此规则无一例外）
- 集体责任：每个成员可以有特长和专注领域，但责任归属整个开发团队
- 无子团队：不包含测试或业务分析等特定领域的子团队

5.2 冲刺与 PDCA 循环

Scrum 的冲刺（Sprint）与戴明环（PDCA / PDSA）有相似之处但不同：

- PDCA 循环由 Walter A. Shewhart 于 1930 年提出，由 W. Edwards. Deming 于 1950 年推广
- PDCA 循环没有严格时长规定，没有规定团队大小
- Scrum 冲刺有固定时间盒（Time-boxed）

[p.89]

5.3 三大制品 + 燃尽图 (Artifacts)

Scrum 过程产生的制品除可工作的软件外，主要有四种： [p.90]

表 15: Scrum 制品

制品	英文	说明
产品待办事项表	Product Backlog	估算单位为故事点（模糊单位），好的产品待办事项表有 DEEP 四个特征（Detailed appropriately, Estimated, Emergent, Prioritized） [p.91]
冲刺待办事项表	Sprint Backlog	定义了开发团队把产品待办事项表条目转换成“完成”增量所需执行的任务；估算单位为小时数， 每天更新 [p.92]
冲刺燃尽图	Sprint Burndown	剩余工作量单位为“人-天数”或“人-时数”，工作日以每周 5 天计；用于跟踪冲刺进度 [p.93]
发布燃尽图	Release Burndown	通过燃尽图确定团队工作情况，了解计划执行情况，用于进度跟踪和改进分析 [p.94]

5.4 四种会议 (Four Meetings / Ceremonies)

Scrum 强调管理，而极限编程强调实践，两者具有很好的互补性。 [p.95]

表 16: Scrum 四种会议

会议	英文	内容与规则
冲刺计划会	Sprint Planning	确定冲刺做什么和怎么做（几小时）。开发团队与产品负责人讨论理解需求,自行确定本次冲刺应完成的产品待办事项条目;然后将条目分解为细粒度任务,形成冲刺待办事项表和冲刺目标。 [p.96]
每日站立会	Daily Stand-up	≤15 分钟,全员参加。三个问题:上次站立会后做了什么?遇到哪些问题?下次站立会前计划做什么? Scrum 主管帮助解决遇到的问题。会后有跟进会议（相关成员进一步沟通）。 [p.99]
冲刺评审会	Sprint Review	≤4 小时（小时数 = 本次冲刺周数）。真实用户和产品负责人检验和使用运行的软件。通过交流审视产品进展,针对问题调整。 [p.100]
冲刺反思会	Sprint Retrospective	冲刺评审会之后。针对流程和环境进行审视和调整。每位成员回顾本次冲刺情况:反思问题和讨论好的工作方式。每位成员对其他成员的反思进行评价,表达期望。 [p.101]

5.5 冲刺计划计算示例

计算开发速度（Velocity）： $\text{已完成的故事点数} (24) \div \text{工作总天数} (53) = 0.45 \text{ 故事点/天}$ [p.97]

计算本次冲刺可完成的故事点数： $\text{开发速度} (0.45 \text{ 故事点/天}) \times \text{团队本次可工作总天数} (49) = 22.05 \text{ 故事点}$ ，取最接近整数 22 个故事点。 [p.98]

6 精益开发与特征驱动开发

行业实际使用的敏捷过程统计：XP 占 8%，Scrum 占 49%，XP+Scrum 结合占 22%，其他敏捷过程占 21%。[p.103]

6.1 精益软件开发（Lean Software Development）

借用精益制造原则形成的敏捷软件过程。精益制造（Lean Manufacturing）是一种优先考虑资源有效利用的生产实践，只关注直接为最终客户贡献价值的活动，同时最大限度减少浪费性支出。[p.104]

表 17: 精益开发七原则

#	原则	英文	内涵
1	整体优化	Optimize Whole	从全局角度优化整个价值流，而非局部优化
2	消除损耗	Eliminate Waste	消除不增加客户价值的活动和官僚主义
3	构建内在品质	Build Quality In	在开发过程中内置质量，而非事后检测修复
4	持续学习	Learn Constantly	通过短周期频繁构建进行学习
5	快速交付	Deliver Fast	快速将价值交付给客户，获取反馈
6	吸引每一个人	Engage Everyone	全员参与，共同推进项目成功
7	越来越好	Keep Getting Better	持续改进，永无止境

[p.104]

6.2 特征驱动软件开发（FDD, Feature Driven Development）

FDD 参考精益制造原则形成的敏捷软件过程。[p.105]

特征（Feature）定义：

- 一个小的、具有客户价值的功能
- “小”：可在两星期之内完成
- “具有客户价值”：可映射到业务过程中某些活动的步骤
- 与 XP 中的用户故事相似，从客户角度理解软件需求

表 18: FDD 的 8 个最佳实践与 5 个子过程

类别	内容
8 个最佳实践	1. 领域对象建模 (Domain Object Modeling) 2. 按特征开发 (Develop by Feature) 3. 个体类 (代码) 拥有权 (Individual Class Ownership) 4. 特征小组 (Feature Teams) 5. 审查 (Inspection) 6. 定期构造 (Regular Builds) 7. 配置管理 (Configuration Management) 8. 结果可视性报告 (Reporting / Visibility of Results)
5 个子过程	1. 开发全局模型 (Develop an Overall Model) 2. 构造特征表 (Build a Feature List) 3. 依据特征制定计划 (Plan by Feature) 4. 特征设计 (Design by Feature) 5. 特征构造 (Build by Feature)

7 敏捷项目管理工具

7.1 PingCode

PingCode 是国内成熟、标准的敏捷开发项目管理软件，适合十几人至几百人等不同规模的敏捷开发团队。 [p.107]

- 完整支持标准的 Scrum 敏捷开发流程、敏捷 Kanban 开发流程以及规模化敏捷管理
- 支持私有部署、二次定制开发和适配国产系统
- 网址: <https://pingcode.com/product/agile>

7.2 Gitee 企业版

Gitee（原名 OSC Git，2013 年正式更名）项目管理覆盖研发管理全生命周期。 [p.115]

- 支持瀑布、敏捷（Scrum、Kanban）等多种开发模型
- 与代码管理、CI/CD、测试管理等功能无缝集成
- 提供强大灵活的自定义配置能力
- 包含：项目创建、成员管理、需求与任务管理、代码仓库（fork/commit）、版本号管理（3 级编码：项目迭代/需求增加/任务迭代）等功能 [p.116–125]
- 网址: <https://gitee.com/>

附录：中英术语对照表

中文	English	缩写
软件过程	Software Process	—
软件生存周期模型	Software Life Cycle Model	—
瀑布模型	Waterfall Model	—
快速原型模型	Prototyping Model	—
快速应用开发	Rapid Application Development	RAD
增量/渐增模型	Incremental Model	—
螺旋模型	Spiral Model	—
统一过程	Unified Process	UP
Rational 统一过程	Rational Unified Process	RUP
统一建模语言	Unified Modeling Language	UML
敏捷开发	Agile Development	—
敏捷联盟宣言	Manifesto of the Agile Alliance	—
极限编程	Extreme Programming	XP
测试驱动开发	Test-Driven Development	TDD
结对编程	Pair Programming	—
重构	Refactoring	—
持续集成	Continuous Integration	CI
用户故事	User Story	—
故事点	Story Point	—
系统隐喻	System Metaphor	—
产品待办事项表	Product Backlog	—
冲刺待办事项表	Sprint Backlog	—
冲刺燃尽图	Sprint Burndown	—
发布燃尽图	Release Burndown	—
冲刺	Sprint	—
站立式会议	Daily Stand-up Meeting	—
冲刺评审会	Sprint Review	—
冲刺反思会	Sprint Retrospective	—
精益软件开发	Lean Software Development	—
特征驱动开发	Feature Driven Development	FDD
专栏:戴明环(计划-执行-检查-处理)	Deming Cycle (Plan-Do-Check-Act)	PDCA